

DATABASE MANAGEMENT SYSTEMS

Group Project Report

AMAZON PRIME

Database Design and Implementation

Memberships | Shopping | Prime Video | Prime Music

Item	Details (fill in before submission)
Course / Subject	Database Management Systems
Submitted to	Prof. Vijendra
Institution	VIT BHOPAL UNIVERSITY
Semester / Year	3 rd semester, 2 nd year
Team member 1	Sousthab Mitra 25BCE10480
Team member 2	Aranyak Sarma 25BCE10994
Team member 3	Chayan Mukherjee 25BCE10620
Team member 4	Ritoriddha Barman 25BCE10854
Date of submission	09-10-2026

Contents

- 1 Abstract
- 2 Introduction and problem statement
- 3 Objectives and scope
- 4 Requirement analysis
- 5 Entities, attributes and relationships
- 6 ER diagrams (conceptual and detailed)
- 7 ER-to-relational mapping
- 8 Data dictionary (24 tables)
- 9 Normalization
- 10 Implementation in SQL
- 11 Sample data
- 12 SQL queries with output (25)
- 13 Transactions and integrity
- 14 Python application
- 15 Analytical charts
- 16 Testing
- 17 Conclusion, limitations and future scope
- 18 References
- 19 Team contribution

1 Abstract

Amazon Prime is a bundled subscription service that combines fast delivery on an online marketplace with video and music streaming. Behind this single membership sit several tightly connected data domains: customers and their payment instruments, membership plans and benefits, a product catalogue with sellers and orders, shipments, reviews, a video library with per-profile viewing history and a music library. This project designs and implements a relational database for such a service. It contains 24 normalized tables, 30 foreign-key relationships, views, triggers and indexes, a deterministic sample data set of about 1,700 rows, 25 analytical SQL queries and a Python console application that performs transactional operations (registration, subscription, ordering, cancellation, viewing). The design was validated with 21 automated tests. The sample data is entirely fictional and the project is an academic model, not Amazon's real schema.

2 Introduction and problem statement

A membership business must answer questions such as: Who is a Prime member today? Which plan earns the most money? Did a Prime customer receive free delivery? Which shows are watched by which profiles? Answering these reliably requires a database that avoids duplicate data, enforces business rules and supports concurrent transactions. Spreadsheets or flat files fail on all three counts.

Problem statement: design and implement a normalized relational database for an Amazon-Prime-like service that (a) stores customers, memberships, payments, shopping and entertainment activity, (b) enforces integrity rules inside the DBMS, and (c) supports operational and analytical queries.

3 Objectives and scope

- Model the real-world domain with an ER diagram that uses 1:1, 1:N, M:N, recursive and exclusive-arc relationships.
- Map the ER model to relational tables in 3NF/BCNF with full key and integrity constraints.
- Implement the schema in MySQL 8 and SQLite using DDL, DML, views, indexes and triggers.
- Write 25 queries covering joins, aggregation, subqueries, CTEs and window functions.
- Build a Python application that uses transactions and demonstrates ACID behaviour.
- Test constraints, triggers and business logic automatically.

In scope: memberships, payments, shopping orders and delivery, reviews, video and music usage. Out of scope: real payment gateways, recommendation algorithms beyond a simple genre rule, a web front-end.

4 Requirement analysis

4.1 Functional requirements

- A customer can register, keep several addresses, and create viewing profiles (including Kids profiles).
- A customer can store payment methods (card, UPI, net banking, wallet).
- A customer can buy one Prime plan (Monthly, Quarterly, Annual, Lite, Shopping Edition, Young Adult) and cancel it.
- Each plan includes a set of benefits (many-to-many).
- Customers browse products from sellers, organised in a category hierarchy, and place orders with several items.
- Prime members receive free delivery and Prime delivery speeds; others pay a fee below Rs 499.
- Every order is paid for and, when dispatched, has one shipment record with tracking and delivery dates.
- Customers who received a product can review it once.
- Profiles watch movies/series (episodes), keep watchlists and play music tracks.
- Management can run reports: revenue, churn, best sellers, late deliveries, popular genres.

4.2 Business rules enforced by the database

#	Rule	Enforced by
1	Email address is unique per customer	UNIQUE constraint
2	Prices > 0, stock >= 0, rating 1-5, quantity > 0	CHECK constraints
3	A payment pays for exactly one subscription OR one order	CHECK exclusive arc
4	A customer has at most one ACTIVE subscription	Trigger trg_one_active_sub
5	Stock reduces when an item is ordered and returns on cancellation	Triggers trg_reduce_stock / trg_restore_stock
6	Overselling is impossible	CHECK(stock_qty >= 0) aborts the transaction
7	Every subscription status change is logged	Trigger trg_subscription_audit
8	A customer reviews a product at most once	UNIQUE(customer_id, product_id)
9	Kids profiles cannot watch 'A'/UA 16+' titles	Application service record_watch()
10	Deleting a customer's profile removes its viewing data	ON DELETE CASCADE

4.3 Non-functional requirements

- Data integrity and consistency (ACID transactions).
- Portability: identical logical design on MySQL and SQLite.
- Performance: indexes on all foreign keys and on frequent filter columns.
- Security: passwords stored only as salted hashes; card/UPI numbers stored masked.

5 Entities, attributes and relationships

The model has five modules. Each coloured group in the detailed ER diagram is one module.

Module	Entities (tables)
Customer & Account	customer, address, profile
Prime Membership	plan, benefit, plan_benefit, subscription, subscription_audit, payment_method, payment
Shopping	seller, category, product, orders, order_item, shipment, review
Prime Video	video_content, episode, watch_history, watchlist
Prime Music	music_artist, music_track, music_play

5.1 Relationships and cardinalities

Entity A	Relationship	Entity B	Cardinality	Implemented as
Customer	has	Address	1:N	foreign key
Customer	owns	Profile	1:N	foreign key
Customer	buys	Subscription	1:N	foreign key
Plan	chosen in	Subscription	1:N	foreign key
Plan	includes	Benefit	M:N	plan_benefit
Subscription	is logged in	Subscription Audit	1:N	foreign key
Customer	registers	Payment Method	1:N	foreign key
Payment Method	is used for	Payment	1:N	foreign key
Subscription	is paid by	Payment	1:N	foreign key
Orders	is paid by	Payment	1:N	foreign key
Customer	places	Orders	1:N	foreign key
Address	is delivered to	Orders	1:N	foreign key
Orders	contains	Product	M:N	order_item
Orders	is fulfilled by	Shipment	1:1	foreign key
Seller	sells	Product	1:N	foreign key
Category	classifies	Product	1:N	foreign key
Category	is parent of	Category	1:N	recursive FK
Customer	writes	Review	1:N	foreign key
Product	receives	Review	1:N	foreign key
Profile	watches	Video Content	M:N	watch_history
Profile	saves to watchlist	Video Content	M:N	watchlist
Video Content	has	Episode	1:N	foreign key
Music Artist	releases	Music Track	1:N	foreign key
Profile	plays	Music Track	M:N	music_play

Special relationship types used: M:N (plan-benefit, order-product, profile-video, profile-track), recursive 1:N (category-parent category), 1:1 (order-shipment) and an exclusive arc (payment pays a subscription or an order).

6 ER diagrams

6.1 Conceptual ER diagram

Rectangles are entities, diamonds are relationships, and the letters show cardinality. Associative tables created from M:N relationships are written in square brackets.

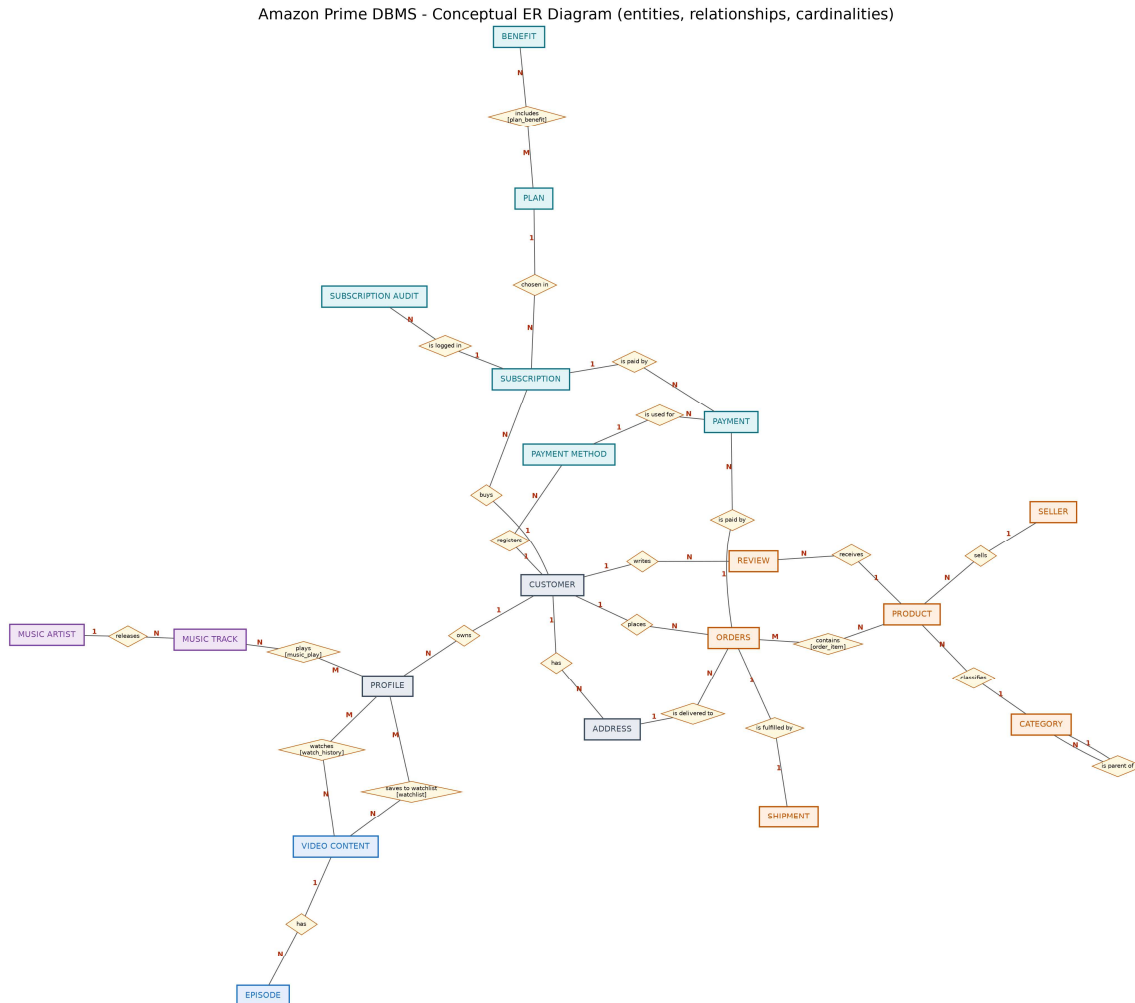


Figure 1 - Conceptual ER diagram

6.2 Detailed ER diagram (crow's-foot)

Every table with all attributes, data types, primary keys (bold, PK), foreign keys (FK), unique columns (U) and nullable columns (?). Vector versions (SVG/PDF) are supplied in the docs folder for zooming.

Amazon Prime DBMS - Detailed ER Diagram (physical model, crow's-foot notation)

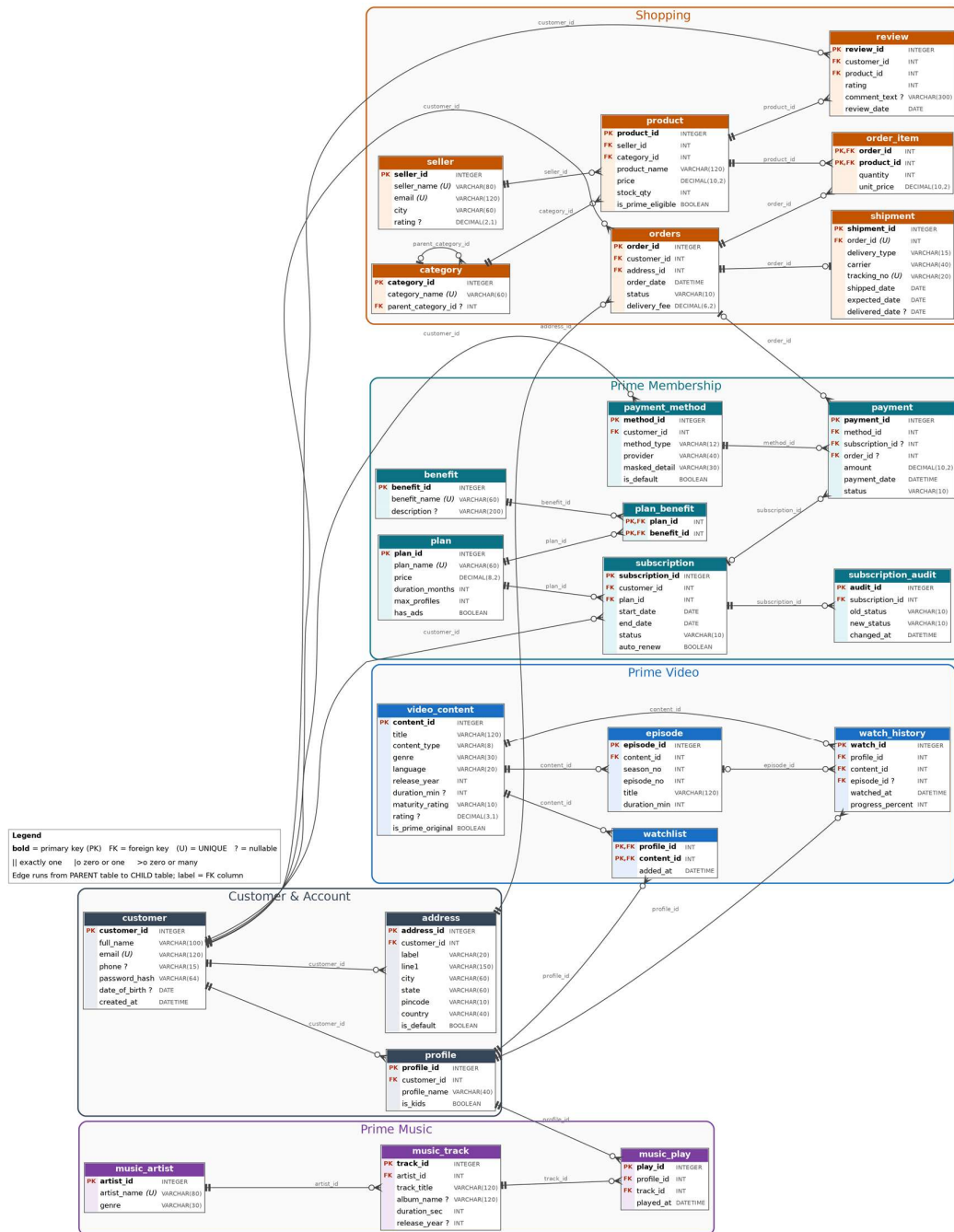


Figure 2 - Detailed ER diagram generated from the actual schema

7 ER-to-relational mapping

ER construct	Mapping rule	Example in this project
Strong entity	Table; key attribute becomes PRIMARY KEY	customer(customer_id, ...)
1:N relationship	Foreign key on the 'N' side	address.customer_id -> customer
M:N relationship	New associative table with composite PK	order_item(order_id, product_id)
M:N with attributes	Attributes go into the associative table	order_item.quantity, unit_price
1:1 relationship	Foreign key + UNIQUE	shipment.order_id UNIQUE
Recursive relationship	Foreign key to the same table	category.parent_category_id
Exclusive arc (OR)	Two nullable FKs + CHECK (exactly one)	payment: subscription_id / order_id
Multivalued attribute	Separate table	Several addresses -> address
Derived attribute	Not stored; computed in a view	Order total -> v_order_totals
Weak/dependent entity	FK + ON DELETE CASCADE	episode depends on video_content

8 Data dictionary

Generated automatically from the live schema. PK = primary key, FK = foreign key, U = unique, NN = NOT NULL.

Customer & Account

8.1 customer

Registered Amazon account holder.

Column	Type	Key	NN	Default
customer_id	INTEGER	PK	Yes	
full_name	VARCHAR(100)		Yes	
email	VARCHAR(120)	U	Yes	
phone	VARCHAR(15)		No	
password_hash	VARCHAR(64)		Yes	
date_of_birth	DATE		No	
created_at	DATETIME		Yes	CURRENT_TIMESTAMP

8.2 address

Delivery addresses of a customer.

Column	Type	Key	NN	Default
address_id	INTEGER	PK	Yes	
customer_id	INT	FK	Yes	
label	VARCHAR(20)		Yes	'Home'
line1	VARCHAR(150)		Yes	
city	VARCHAR(60)		Yes	
state	VARCHAR(60)		Yes	
pincode	VARCHAR(10)		Yes	
country	VARCHAR(40)		Yes	'India'
is_default	BOOLEAN		Yes	0

8.3 profile

Viewing profile inside an account (adult or kids).

Column	Type	Key	NN	Default
profile_id	INTEGER	PK	Yes	
customer_id	INT	FK	Yes	
profile_name	VARCHAR(40)		Yes	
is_kids	BOOLEAN		Yes	0

Prime Membership

8.4 plan

Prime membership plans and prices.

Column	Type	Key	NN	Default
plan_id	INTEGER	PK	Yes	
plan_name	VARCHAR(60)	U	Yes	
price	DECIMAL(8,2)		Yes	
duration_months	INT		Yes	
max_profiles	INT		Yes	6
has_ads	BOOLEAN		Yes	0

8.5 benefit

Individual Prime benefits.

Column	Type	Key	NN	Default
benefit_id	INTEGER	PK	Yes	
benefit_name	VARCHAR(60)	U	Yes	
description	VARCHAR(200)		No	

8.6 plan_benefit

Which benefits each plan includes (M:N).

Column	Type	Key	NN	Default
plan_id	INT	PK, FK	Yes	
benefit_id	INT	PK, FK	Yes	

8.7 subscription

A customer's membership period for a plan.

Column	Type	Key	NN	Default
subscription_id	INTEGER	PK	Yes	
customer_id	INT	FK	Yes	
plan_id	INT	FK	Yes	
start_date	DATE		Yes	
end_date	DATE		Yes	
status	VARCHAR(10)		Yes	'ACTIVE'
auto_renew	BOOLEAN		Yes	1

8.8 subscription_audit

Trigger-written log of status changes.

Column	Type	Key	NN	Default
audit_id	INTEGER	PK	Yes	
subscription_id	INT	FK	Yes	
old_status	VARCHAR(10)		Yes	
new_status	VARCHAR(10)		Yes	
changed_at	DATETIME		Yes	CURRENT_TIMESTAMP

8.9 payment_method

Stored (masked) payment instruments.

Column	Type	Key	NN	Default
method_id	INTEGER	PK	Yes	
customer_id	INT	FK	Yes	
method_type	VARCHAR(12)		Yes	
provider	VARCHAR(40)		Yes	
masked_detail	VARCHAR(30)		Yes	
is_default	BOOLEAN		Yes	0

8.10 payment

Money received for a subscription or an order.

Column	Type	Key	NN	Default
payment_id	INTEGER	PK	Yes	
method_id	INT	FK	Yes	
subscription_id	INT	FK	No	
order_id	INT	FK	No	
amount	DECIMAL(10,2)		Yes	
payment_date	DATETIME		Yes	CURRENT_TIMESTAMP
status	VARCHAR(10)		Yes	'SUCCESS'

Shopping

8.11 seller

Third-party sellers on the marketplace.

Column	Type	Key	NN	Default
seller_id	INTEGER	PK	Yes	
seller_name	VARCHAR(80)	U	Yes	
email	VARCHAR(120)	U	Yes	
city	VARCHAR(60)		Yes	
rating	DECIMAL(2,1)		No	

8.12 category

Product categories (self-referencing hierarchy).

Column	Type	Key	NN	Default
category_id	INTEGER	PK	Yes	
category_name	VARCHAR(60)	U	Yes	
parent_category_id	INT	FK	No	

8.13 product

Items for sale.

Column	Type	Key	NN	Default
product_id	INTEGER	PK	Yes	
seller_id	INT	FK	Yes	
category_id	INT	FK	Yes	
product_name	VARCHAR(120)		Yes	
price	DECIMAL(10,2)		Yes	
stock_qty	INT		Yes	0
is_prime_eligible	BOOLEAN		Yes	1

8.14 orders

Customer orders (header).

Column	Type	Key	NN	Default
order_id	INTEGER	PK	Yes	
customer_id	INT	FK	Yes	
address_id	INT	FK	Yes	
order_date	DATETIME		Yes	CURRENT_TIMESTAMP
status	VARCHAR(10)		Yes	'PLACED'
delivery_fee	DECIMAL(6,2)		Yes	0

8.15 order_item

Order lines with price at purchase time.

Column	Type	Key	NN	Default
order_id	INT	PK, FK	Yes	
product_id	INT	PK, FK	Yes	
quantity	INT		Yes	
unit_price	DECIMAL(10,2)		Yes	

8.16 shipment

Dispatch and delivery details (1:1 with order).

Column	Type	Key	NN	Default
shipment_id	INTEGER	PK	Yes	
order_id	INT	FK, U	Yes	
delivery_type	VARCHAR(15)		Yes	
carrier	VARCHAR(40)		Yes	
tracking_no	VARCHAR(20)	U	Yes	
shipped_date	DATE		Yes	
expected_date	DATE		Yes	
delivered_date	DATE		No	

8.17 review

Product reviews.

Column	Type	Key	NN	Default
review_id	INTEGER	PK	Yes	
customer_id	INT	FK	Yes	
product_id	INT	FK	Yes	
rating	INT		Yes	
comment_text	VARCHAR(300)		No	
review_date	DATE		Yes	

Prime Video

8.18 video_content

Movies and series on Prime Video.

Column	Type	Key	NN	Default
content_id	INTEGER	PK	Yes	
title	VARCHAR(120)		Yes	
content_type	VARCHAR(8)		Yes	
genre	VARCHAR(30)		Yes	
language	VARCHAR(20)		Yes	
release_year	INT		Yes	
duration_min	INT		No	
maturity_rating	VARCHAR(10)		Yes	
rating	DECIMAL(3,1)		No	
is_prime_original	BOOLEAN		Yes	0

8.19 episode

Episodes of a series.

Column	Type	Key	NN	Default
episode_id	INTEGER	PK	Yes	
content_id	INT	FK	Yes	
season_no	INT		Yes	
episode_no	INT		Yes	
title	VARCHAR(120)		Yes	
duration_min	INT		Yes	

8.20 watch_history

Which profile watched what and when.

Column	Type	Key	NN	Default
watch_id	INTEGER	PK	Yes	
profile_id	INT	FK	Yes	
content_id	INT	FK	Yes	
episode_id	INT	FK	No	
watched_at	DATETIME		Yes	
progress_percent	INT		Yes	0

8.21 watchlist

Titles saved by a profile (M:N).

Column	Type	Key	NN	Default
profile_id	INT	PK, FK	Yes	
content_id	INT	PK, FK	Yes	
added_at	DATETIME		Yes	CURRENT_TIMESTAMP

Prime Music

8.22 music_artist

Music artists.

Column	Type	Key	NN	Default
artist_id	INTEGER	PK	Yes	
artist_name	VARCHAR(80)	U	Yes	
genre	VARCHAR(30)		Yes	

8.23 music_track

Songs.

Column	Type	Key	NN	Default
track_id	INTEGER	PK	Yes	
artist_id	INT	FK	Yes	
track_title	VARCHAR(120)		Yes	
album_name	VARCHAR(120)		No	
duration_sec	INT		Yes	
release_year	INT		No	

8.24 music_play

Play events per profile.

Column	Type	Key	NN	Default
play_id	INTEGER	PK	Yes	
profile_id	INT	FK	Yes	
track_id	INT	FK	Yes	
played_at	DATETIME		Yes	

9 Normalization

All tables were normalized to at least Third Normal Form (most are in BCNF). The reasoning is below.

9.1 Starting point: an un-normalized order sheet

```
ORDER_SHEET(order_id, order_date, customer_name, customer_email, customer_phone,  
            delivery_address, product_ids, product_names, seller_names, quantities,  
            prices, payment_type, tracking_no, plan_name, plan_price, plan_benefits)
```

Problems: repeating groups (several products and benefits in one row), partial and transitive dependencies, and insert/update/delete anomalies.

9.2 Steps

Normal form	Problem removed	Result
1NF	Repeating groups (products, benefits) and multivalued data (addresses) removed; every column atomic.	order_item, plan_benefit, address
2NF	Partial dependencies on part of a composite key removed. In (order_id, product_id) the product name depends only on product_id.	product separated from order_item
3NF	Transitive dependencies removed: product -> seller -> seller_city; plan_id -> price; customer_id -> email.	seller, plan, customer separated
BCNF	Every determinant is a candidate key. Check: in subscription, subscription_id -> all; (customer_id, start_date) is a candidate key as well, no other determinants.	No further split needed

9.3 Functional dependencies (examples)

```
customer_id  -> full_name, email, phone, password_hash, date_of_birth, created_at  
email        -> customer_id (candidate key)  
plan_id      -> plan_name, price, duration_months, max_profiles, has_ads  
(order_id, product_id) -> quantity, unit_price  
order_id     -> customer_id, address_id, order_date, status, delivery_fee  
product_id   -> seller_id, category_id, product_name, price, stock_qty  
category_id  -> category_name, parent_category_id  
subscription_id -> customer_id, plan_id, start_date, end_date, status, auto_renew  
(profile_id, content_id) -> added_at (watchlist)
```

9.4 Deliberate design decisions

- **unit_price in order_item:** looks redundant with product.price but is NOT - it records the price at purchase time, which changes later. It is history, not duplication.
- **No stored order total:** it is a derived value, so it lives in the view v_order_totals (avoids update anomalies).
- **Selective denormalization avoided:** no counters or totals are stored in base tables.

10 Implementation in SQL

Two equivalent script sets are supplied: sql/mysql (MySQL 8.0+) and sql/sqlite (SQLite 3.35+). They are generated from one template, so the logical design is identical. Order of execution: 01_schema.sql, 02_sample_data.sql, 03_views_triggers.sql, 04_queries.sql.

10.1 Example DDL

```
CREATE TABLE subscription (  
  subscription_id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id    INT          NOT NULL,  
  plan_id        INT          NOT NULL,  
  start_date     DATE         NOT NULL,  
  end_date       DATE         NOT NULL,  
  status         VARCHAR(10) NOT NULL DEFAULT 'ACTIVE'  
                CHECK (status IN ('ACTIVE', 'EXPIRED', 'CANCELLED')),  
  auto_renew     BOOLEAN      NOT NULL DEFAULT 1,  
  CHECK (end_date > start_date),  
  FOREIGN KEY (customer_id) REFERENCES customer(customer_id),  
  FOREIGN KEY (plan_id)     REFERENCES plan(plan_id)  
) ENGINE=InnoDB;
```

10.2 Constraint summary

Object	Count
Tables	24
Foreign keys	30
UNIQUE columns/sets	14
CHECK constraints	29
Indexes (explicit)	14
Views	3
Triggers	4

10.2 Views

- **v_order_totals** - items total + delivery fee = grand total per order.
- **v_active_subscribers** - customers whose membership is currently ACTIVE.
- **v_product_ratings** - review count and average rating per product.

10.3 Triggers (MySQL version shown)

```
CREATE TRIGGER trg_one_active_sub BEFORE INSERT ON subscription  
FOR EACH ROW  
BEGIN  
  IF NEW.status = 'ACTIVE' AND EXISTS (SELECT 1 FROM subscription  
                                       WHERE customer_id = NEW.customer_id  
                                       AND status = 'ACTIVE') THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Customer already has an ACTIVE subscription';  
  END IF;  
END
```

```
CREATE TRIGGER trg_reduce_stock AFTER INSERT ON order_item  
FOR EACH ROW  
BEGIN  
  UPDATE product SET stock_qty = stock_qty - NEW.quantity  
  WHERE product_id = NEW.product_id;  
END
```

10.4 Indexing

Indexes were added on every foreign key used in joins (e.g. orders.customer_id, order_item.product_id, watch_history.profile_id) and on columns used in filters (subscription(status, end_date), orders.order_date). Primary and UNIQUE keys are indexed automatically.

11 Sample data

Data is generated by scripts/generate_data.py with a fixed random seed, so the same rows are produced on every run. All names, e-mails, ratings and activity are fictional. Two products are intentionally never ordered and two kids profiles intentionally watched adult titles, so that queries Q19 and Q14 have something to find.

Table	Rows
customer	40
address	50
profile	71
plan	6
benefit	7
plan_benefit	34
subscription	72
subscription_audit	4
payment_method	78
payment	162
seller	8
category	19
product	36
orders	90
order_item	199
shipment	83
review	97
video_content	20
episode	72
watch_history	336
watchlist	70
music_artist	7
music_track	17
music_play	160
TOTAL	1738

12 SQL queries with output

All queries are in `sql/04_queries.sql`. Outputs below were produced by running them on the sample data (first 6 rows shown). Sample 'today' is 2026-10-05.

Q1 Active Prime members with their plan and renewal date (JOIN via view)

Concepts: view, JOIN

```
SELECT full_name, plan_name, start_date, end_date, auto_renew
FROM   v_active_subscribers
ORDER BY end_date;
```

full_name	plan_name	start_date	end_date	auto_renew
Sneha Kulkarni	Prime Annual	2025-10-17	2026-10-17	0
Pooja Gupta	Prime Young Adult Annual	2025-11-01	2026-11-01	0
Aditya Das	Prime Young Adult Annual	2025-11-07	2026-11-07	1
Isha Roy	Prime Annual	2025-11-29	2026-11-29	0
Sourav Joshi	Prime Annual	2025-12-03	2026-12-03	1
Priya Mukherjee	Prime Young Adult Annual	2026-01-04	2027-01-04	1

... 17 rows in total

Q2 Number of subscriptions per plan (GROUP BY)

Concepts: GROUP BY

```
SELECT p.plan_name, COUNT(*) AS total_subscriptions
FROM   subscription s
JOIN   plan p ON p.plan_id = s.plan_id
GROUP BY p.plan_name
ORDER BY total_subscriptions DESC;
```

plan_name	total_subscriptions
Prime Annual	24
Prime Monthly	16
Prime Young Adult Annual	10
Prime Lite Annual	8
Prime Shopping Edition	7
Prime Quarterly	7

Q3 Net subscription revenue per plan (refunded payments excluded)

Concepts: multi-table JOIN, SUM

```
SELECT p.plan_name, SUM(pay.amount) AS net_revenue
FROM   payment pay
JOIN   subscription s ON s.subscription_id = pay.subscription_id
JOIN   plan p ON p.plan_id = s.plan_id
WHERE  pay.status = 'SUCCESS'
GROUP BY p.plan_name
ORDER BY net_revenue DESC;
```

plan_name	net_revenue
Prime Annual	31479
Prime Young Adult Annual	6741
Prime Lite Annual	6392
Prime Monthly	4784
Prime Quarterly	4193
Prime Shopping Edition	2793

Q4 Customers who have NEVER subscribed to Prime (anti-join with NOT EXISTS)

Concepts: NOT EXISTS

```
SELECT c.customer_id, c.full_name, c.email
FROM   customer c
WHERE  NOT EXISTS (SELECT 1 FROM subscription s WHERE s.customer_id = c.customer_id)
ORDER BY c.customer_id;
```

customer_id	full_name	email
33	Tushar Chatterjee	tushar.chatterjee33@example.com
34	Ishaan Kulkarni	ishaan.kulkarni34@example.com
35	Lakshmi Singh	lakshmi.singh35@example.com
36	Farhan Patel	farhan.patel36@example.com
37	Zoya Nair	zoya.nair37@example.com
38	Harsh Nair	harsh.nair38@example.com

... 8 rows in total

Q5 Active subscriptions expiring in the next 30 days (renewal campaign list)

Concepts: date range

```
SELECT c.full_name, c.phone, p.plan_name, s.end_date, s.auto_renew
FROM   subscription s
JOIN   customer c ON c.customer_id = s.customer_id
JOIN   plan p     ON p.plan_id = s.plan_id
WHERE  s.status = 'ACTIVE'
AND    s.end_date BETWEEN '2026-10-05' AND '2026-11-04'
ORDER BY s.end_date;
```

full_name	phone	plan_name	end_date	auto_renew
Sneha Kulkarni	9196277024	Prime Annual	2026-10-17	0
Pooja Gupta	9229728319	Prime Young Adult Annual	2026-11-01	0

Q6 Top 5 best-selling products by units sold (cancelled/returned excluded)

Concepts: JOIN, ORDER BY, LIMIT

```
SELECT pr.product_name, SUM(oi.quantity) AS units_sold
FROM   order_item oi
JOIN   orders o ON o.order_id = oi.order_id
JOIN   product pr ON pr.product_id = oi.product_id
WHERE  o.status NOT IN ('CANCELLED', 'RETURNED')
GROUP BY pr.product_id, pr.product_name
ORDER BY units_sold DESC, pr.product_name
LIMIT 5;
```

product_name	units_sold
Non-stick Cookware Set (5 pcs)	16
Electric Kettle 1.5L	13
Women's Printed Kurti	13
Wireless Mouse	12
Laptop Backpack 30L	11

Q7 Revenue per top-level category (self-join on the category hierarchy)

Concepts: self-join hierarchy

```
SELECT COALESCE(par.category_name, cat.category_name) AS top_category,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM   order_item oi
JOIN   orders o ON o.order_id = oi.order_id
JOIN   product pr ON pr.product_id = oi.product_id
JOIN   category cat ON cat.category_id = pr.category_id
```

```

LEFT JOIN category par ON par.category_id = cat.parent_category_id
WHERE o.status NOT IN ('CANCELLED','RETURNED')
GROUP BY COALESCE(par.category_name, cat.category_name)
ORDER BY revenue DESC;

```

top_category	revenue
Electronics	1135749
Home & Kitchen	72855
Fashion	60157
Books	27911
Grocery	19426
Sports & Fitness	1797

Q8 Products with at least 3 reviews, ranked by average rating (HAVING)

Concepts: HAVING

```

SELECT pr.product_name, COUNT(r.review_id) AS reviews, ROUND(AVG(r.rating), 2) AS avg_rating
FROM product pr
JOIN review r ON r.product_id = pr.product_id
GROUP BY pr.product_id, pr.product_name
HAVING COUNT(r.review_id) >= 3
ORDER BY avg_rating DESC, reviews DESC;

```

product_name	reviews	avg_rating
Stainless Steel Lunch Box	3	4.67
Fundamentals of Database Systems (7th Ed.)	3	4.33
Women's Printed Kurti	4	4.25
Running Shoes	6	4.17
Casual Sneakers	4	4.00
Electric Kettle 1.5L	3	4.00

... 17 rows in total

Q9 Average order value - Prime delivery vs standard delivery (CASE + view)

Concepts: CASE, view

```

SELECT CASE WHEN sh.delivery_type = 'STANDARD' THEN 'Standard delivery'
           ELSE 'Prime delivery' END AS delivery_group,
COUNT(*) AS orders,
ROUND(AVG(t.grand_total), 2) AS avg_order_value
FROM v_order_totals t
JOIN shipment sh ON sh.order_id = t.order_id
GROUP BY CASE WHEN sh.delivery_type = 'STANDARD' THEN 'Standard delivery'
           ELSE 'Prime delivery' END;

```

delivery_group	orders	avg_order_value
Prime delivery	39	24,799.21
Standard delivery	44	16,358.45

Q10 Late deliveries and how many days late

Concepts: date arithmetic

```

SELECT o.order_id, c.full_name, sh.delivery_type, sh.expected_date, sh.delivered_date,
CAST(julianday(sh.delivered_date) - julianday(sh.expected_date) AS INTEGER) AS days_late
FROM shipment sh
JOIN orders o ON o.order_id = sh.order_id
JOIN customer c ON c.customer_id = o.customer_id
WHERE sh.delivered_date > sh.expected_date
ORDER BY days_late DESC, o.order_id;

```

order_id	full_name	delivery_type	expected_date	delivered_date	days_late
7	Arjun Reddy	PRIME_ONE_DAY	2026-09-04	2026-09-06	2
20	Aditya Das	PRIME_ONE_DAY	2025-07-23	2025-07-25	2
21	Sourav Joshi	PRIME_ONE_DAY	2026-04-30	2026-05-02	2
22	Sneha Kulkarni	STANDARD	2025-05-27	2025-05-29	2
32	Nikhil Chatterjee	STANDARD	2025-06-10	2025-06-12	2
34	Siddharth Joshi	STANDARD	2025-08-27	2025-08-29	2

... 25 rows in total

Q11 Customers who spent more than the average customer (scalar subquery)

Concepts: scalar subquery

```
SELECT c.full_name, ROUND(SUM(t.grand_total), 2) AS total_spent
FROM   v_order_totals t
JOIN   customer c ON c.customer_id = t.customer_id
WHERE  t.status NOT IN ('CANCELLED', 'RETURNED')
GROUP BY c.customer_id, c.full_name
HAVING SUM(t.grand_total) > (SELECT AVG(x.s) FROM
    (SELECT SUM(grand_total) AS s FROM v_order_totals
     WHERE status NOT IN ('CANCELLED', 'RETURNED') GROUP BY customer_id) x)
ORDER BY total_spent DESC;
```

full_name	total_spent
Arjun Reddy	220,451.00
Mitali Roy	158,705.00
Ananya Malhotra	131,015.00
Tushar Chatterjee	113,074.00
Neha Patel	92,375.00
Anjali Banerjee	86,278.00

... 10 rows in total

Q12 Ten most-watched titles (distinct profiles that watched them)

Concepts: COUNT DISTINCT

```
SELECT v.title, v.content_type, COUNT(*) AS views, COUNT(DISTINCT w.profile_id) AS unique_viewers
FROM   watch_history w
JOIN   video_content v ON v.content_id = w.content_id
GROUP BY v.content_id, v.title, v.content_type
ORDER BY views DESC, v.title
LIMIT 10;
```

title	content_type	views	unique_viewers
Mirzapur	SERIES	34	22
Panchayat	SERIES	34	25
Gulabo Sitabo	MOVIE	30	21
Jai Bhim	MOVIE	28	21
The Family Man	SERIES	24	17
The Marvelous Mrs. Maisel	SERIES	22	20

... 10 rows in total

Q13 Genre popularity on Prime Video

Concepts: AVG

```
SELECT v.genre, COUNT(*) AS views, ROUND(AVG(w.progress_percent), 1) AS avg_completion_pct
FROM   watch_history w
JOIN   video_content v ON v.content_id = w.content_id
GROUP BY v.genre
ORDER BY views DESC;
```

genre	views	avg_completion_pct
Comedy	105	69.40
Drama	65	65.40
Crime	39	69.40
Animation	35	69.90
Action	30	67.80
Thriller	24	86.70

... 9 rows in total

Q14 Parental-control audit - kids profiles that watched 'A' (adult) content

Concepts: multi-join audit

```
SELECT c.full_name AS account_holder, pf.profile_name, v.title, v.maturity_rating, w.watched_at
FROM   watch_history w
JOIN   profile pf      ON pf.profile_id = w.profile_id
JOIN   customer c      ON c.customer_id = pf.customer_id
JOIN   video_content v ON v.content_id  = w.content_id
WHERE  pf.is_kids = 1 AND v.maturity_rating = 'A';
```

account_holder	profile_name	title	maturity_rating	watched_at
Tanvi Joshi	Kids	Paatal Lok	A	2026-09-15 21:40:14
Aarav Mukherjee	Kids	Mirzapur	A	2026-09-15 08:56:05

Q15 Each profile's most-watched title (window function RANK)

Concepts: window RANK

```
SELECT profile_name, title, views FROM (
  SELECT pf.profile_name, v.title, COUNT(*) AS views,
         RANK() OVER (PARTITION BY pf.profile_id ORDER BY COUNT(*) DESC) AS rnk
  FROM   watch_history w
  JOIN   profile pf      ON pf.profile_id = w.profile_id
  JOIN   video_content v ON v.content_id  = w.content_id
  GROUP BY pf.profile_id, pf.profile_name, v.content_id, v.title
) ranked
WHERE rnk = 1 AND views >= 3
ORDER BY views DESC, profile_name;
```

profile_name	title	views
Kids	Gulabo Sitabo	4
Family	Jai Bhim	3
Family	The Boys	3
Kids	Kids Cartoon Carnival	3
Kids	Little Explorers	3
Kids	Kids Cartoon Carnival	3

... 13 rows in total

Q16 Monthly revenue and running total (CTE + window SUM OVER)

Concepts: CTE + running SUM

```
WITH monthly AS (
  SELECT strftime('%Y-%m', order_date) AS month, SUM(grand_total) AS revenue
  FROM   v_order_totals
  WHERE  status NOT IN ('CANCELLED', 'RETURNED')
  GROUP BY strftime('%Y-%m', order_date)
)
SELECT month, ROUND(revenue, 2) AS revenue,
       ROUND(SUM(revenue) OVER (ORDER BY month), 2) AS running_total
FROM   monthly
ORDER BY month;
```

month	revenue	running_total
2025-01	79,212.00	79,212.00
2025-02	6,193.00	85,405.00
2025-03	17,841.00	103,246.00
2025-04	202,804.00	306,050.00
2025-05	55,836.00	361,886.00
2025-06	90,075.00	451,961.00

... 21 rows in total

Q17 Rank customers by spend inside each city (PARTITION BY)

Concepts: RANK PARTITION BY

```
SELECT city, full_name, total_spent, city_rank FROM (
  SELECT a.city, c.full_name, ROUND(SUM(t.grand_total), 2) AS total_spent,
         RANK() OVER (PARTITION BY a.city ORDER BY SUM(t.grand_total) DESC) AS city_rank
  FROM   v_order_totals t
  JOIN   orders o    ON o.order_id = t.order_id
  JOIN   address a    ON a.address_id = o.address_id
  JOIN   customer c   ON c.customer_id = t.customer_id
  WHERE  t.status NOT IN ('CANCELLED', 'RETURNED')
  GROUP BY a.city, c.customer_id, c.full_name
) r
WHERE city_rank <= 2
ORDER BY city, city_rank;
```

city	full_name	total_spent	city_rank
Ahmedabad	Rohan Sen	72,878.00	1
Ahmedabad	Amit Mehta	2,348.00	2
Bengaluru	Ananya Malhotra	131,015.00	1
Bengaluru	Meera Sharma	32,088.00	2
Chennai	Vikram Banerjee	13,493.00	1
Chennai	Tanvi Joshi	9,541.00	2

... 18 rows in total

Q18 Active Prime members who placed 3 or more orders (CTE + join)

Concepts: CTE

```
WITH order_counts AS (
  SELECT customer_id, COUNT(*) AS n_orders FROM orders GROUP BY customer_id
)
SELECT v.full_name, v.plan_name, oc.n_orders
FROM   v_active_subscribers v
JOIN   order_counts oc ON oc.customer_id = v.customer_id
WHERE  oc.n_orders >= 3
ORDER BY oc.n_orders DESC, v.full_name;
```

full_name	plan_name	n_orders
Mitali Roy	Prime Lite Annual	5
Sourav Joshi	Prime Annual	5
Tanvi Joshi	Prime Annual	5
Sneha Kulkarni	Prime Annual	4
Anjali Banerjee	Prime Annual	3
Meera Sharma	Prime Young Adult Annual	3

... 7 rows in total

Q19 Products that were never ordered (LEFT JOIN ... IS NULL)

Concepts: LEFT JOIN IS NULL

```
SELECT pr.product_id, pr.product_name, pr.stock_qty
FROM   product pr
LEFT JOIN order_item oi ON oi.product_id = pr.product_id
```

WHERE oi.product_id IS NULL;

product_id	product_name	stock_qty
34	Adjustable Dumbbell Set 20kg	70
35	Skipping Rope	450

Q20 Seller scorecard - revenue and average customer rating

Concepts: derived tables

```
SELECT s.seller_name,
       COALESCE(rev.revenue, 0) AS revenue,
       COALESCE(rat.avg_rating, 0) AS avg_product_rating
FROM   seller s
LEFT JOIN (SELECT pr.seller_id, SUM(oi.quantity * oi.unit_price) AS revenue
           FROM order_item oi
           JOIN orders o ON o.order_id = oi.order_id
           JOIN product pr ON pr.product_id = oi.product_id
           WHERE o.status NOT IN ('CANCELLED', 'RETURNED')
           GROUP BY pr.seller_id) rev ON rev.seller_id = s.seller_id
LEFT JOIN (SELECT pr.seller_id, ROUND(AVG(r.rating), 2) AS avg_rating
           FROM review r JOIN product pr ON pr.product_id = r.product_id
           GROUP BY pr.seller_id) rat ON rat.seller_id = s.seller_id
ORDER BY revenue DESC;
```

seller_name	revenue	avg_product_rating
TechZone Traders	981885	3.55
Prime Retail Pvt Ltd	89185	3.67
HomeEssentials Co	72855	3.87
GadgetGalaxy	64679	2.80
StyleHub Fashions	60157	3.95
BookNest Publishers	27911	3.40

... 8 rows in total

Q21 Most played Prime Music artists

Concepts: JOIN, GROUP BY

```
SELECT ar.artist_name, COUNT(*) AS plays
FROM   music_play mp
JOIN   music_track t ON t.track_id = mp.track_id
JOIN   music_artist ar ON ar.artist_id = t.artist_id
GROUP BY ar.artist_id, ar.artist_name
ORDER BY plays DESC, ar.artist_name;
```

artist_name	plays
Arijit Singh	46
Taylor Swift	28
A. R. Rahman	26
Ed Sheeran	22
Coldplay	20
Queen	17

... 7 rows in total

Q22 Customers who use more than one type of payment method

Concepts: GROUP_CONCAT

```
SELECT c.full_name, COUNT(DISTINCT pm.method_type) AS method_types,
       GROUP_CONCAT(pm.provider, ', ') AS providers
FROM   customer c
JOIN   payment_method pm ON pm.customer_id = c.customer_id
GROUP BY c.customer_id, c.full_name
HAVING COUNT(DISTINCT pm.method_type) > 1
```

```
ORDER BY method_types DESC, c.full_name
LIMIT 10;
```

full_name	method_types	providers
Aditya Das	3	SBI Mastercard, Amazon Pay Balance, State Bank of India
Debashis Gupta	3	Amazon Pay Balance, Axis RuPay, Google Pay
Vikram Banerjee	3	SBI Mastercard, Amazon Pay Balance, State Bank of India
Aarav Mukherjee	2	Amazon Pay Balance, SBI Mastercard
Amit Mehta	2	Google Pay, Amazon Pay Balance, PhonePe
Anjali Banerjee	2	Paytm, Amazon Pay Balance

... 10 rows in total

Q23 Subscription churn - share of subscriptions cancelled, per plan

Concepts: conditional aggregation

```
SELECT p.plan_name,
       COUNT(*) AS subscriptions,
       SUM(CASE WHEN s.status = 'CANCELLED' THEN 1 ELSE 0 END) AS cancelled,
       ROUND(100.0 * SUM(CASE WHEN s.status = 'CANCELLED' THEN 1 ELSE 0 END) / COUNT(*), 1) AS churn_pct
FROM   subscription s
JOIN   plan p ON p.plan_id = s.plan_id
GROUP BY p.plan_name
ORDER BY churn_pct DESC, p.plan_name;
```

plan_name	subscriptions	cancelled	churn_pct
Prime Annual	24	3	12.50
Prime Young Adult Annual	10	1	10.00
Prime Lite Annual	8	0	0.00
Prime Monthly	16	0	0.00
Prime Quarterly	7	0	0.00
Prime Shopping Edition	7	0	0.00

Q24 Benefits included in each plan (M:N resolved with GROUP_CONCAT)

Concepts: M:N + GROUP_CONCAT

```
SELECT p.plan_name, p.price, COUNT(b.benefit_id) AS benefit_count,
       GROUP_CONCAT(b.benefit_name, ', ') AS benefits
FROM   plan p
JOIN   plan_benefit pb ON pb.plan_id = p.plan_id
JOIN   benefit b ON b.benefit_id = pb.benefit_id
GROUP BY p.plan_id, p.plan_name, p.price
ORDER BY p.price DESC;
```

plan_name	price	benefit_count	benefits
Prime Annual	1499	7	Free & Fast Delivery, Prime Video, Prime Music, Prime Reading, Early Access to Deals, Ad-free Streaming, Prime Gaming
Prime Lite Annual	799	4	Free & Fast Delivery, Prime Video, Prime Music, Early Access to Deals
Prime Young Adult Annual	749	7	Free & Fast Delivery, Prime Video, Prime Music, Prime Reading, Early Access to Deals, Ad-free Streaming, Prime Gaming
Prime Quarterly	599	7	Free & Fast Delivery, Prime Video, Prime Music, Prime Reading, Early Access to Deals,

			Ad-free Streaming, Prime Gaming
Prime Shopping Edition	399	2	Free & Fast Delivery, Early Access to Deals
Prime Monthly	299	7	Free & Fast Delivery, Prime Video, Prime Music, Prime Reading, Early Access to Deals, Ad-free Streaming, Prime Gaming

Q25 Engagement score per account = video views + music plays (UNION ALL)

Concepts: UNION ALL

```
SELECT c.full_name, SUM(a.events) AS engagement_score
FROM (
  SELECT pf.customer_id, COUNT(*) AS events
  FROM watch_history w JOIN profile pf ON pf.profile_id = w.profile_id GROUP BY pf.customer_id
  UNION ALL
  SELECT pf.customer_id, COUNT(*) AS events
  FROM music_play m JOIN profile pf ON pf.profile_id = m.profile_id GROUP BY pf.customer_id
) a
JOIN customer c ON c.customer_id = a.customer_id
GROUP BY c.customer_id, c.full_name
ORDER BY engagement_score DESC
LIMIT 10;
```

full_name	engagement_score
Manish Mukherjee	28
Meera Sharma	25
Vikram Banerjee	24
Shreya Roy	24
Tanvi Joshi	23
Isha Roy	22

... 10 rows in total

13 Transactions and integrity

Operations that touch several tables run inside a single transaction so that the database never shows a half-finished business action (atomicity) and always moves from one valid state to another (consistency).

Operation	Tables changed in ONE transaction	Failure behaviour
Register customer	customer, address, profile	Duplicate e-mail -> nothing saved
Subscribe to Prime	subscription, payment (+ audit via trigger)	Second ACTIVE plan -> rollback
Place order	orders, order_item (+ stock trigger), payment	Insufficient stock -> whole order rolled back
Cancel order	orders (+ stock restore trigger), payment	Only PLACED orders can be cancelled
Cancel membership	subscription (+ audit), payment	No active plan -> error

Isolation and durability are provided by the DBMS (InnoDB / SQLite journal). A real deployment would use SELECT ... FOR UPDATE on product rows when many users order the same item concurrently.

14 Python application

The console application (app/) uses only the Python standard library.

File	Role
db.py	Connection, foreign-key enforcement, database (re)creation, table printer
services.py	Business logic: register, subscribe, cancel, order, ship, review, watch, recommend
prime_cli.py	Interactive menu and a scripted --demo walkthrough
reports.py	Row counts and matplotlib charts

Run: `python app/prime_cli.py --reset` then `python app/prime_cli.py --demo`

14.1 Demo output (extract)

```
3. Order BEFORE Prime: delivery fee applies below Rs 499
   {'order_id': 91, 'items_total': 249.0, 'delivery_fee': 40.0, 'grand_total': 289.0, 'prime_member':
False}
5. Business rule: a second ACTIVE plan is rejected by a TRIGGER
   Rejected -> Customer already has an ACTIVE subscription
6. Order AFTER Prime: free delivery
   {'order_id': 92, 'items_total': 2248.0, 'delivery_fee': 0.0, 'grand_total': 2248.0, 'prime_member':
True}
7. Overselling is blocked and the whole order is rolled back
   Rejected -> Only 70 unit(s) of 'Adjustable Dumbbell Set 20kg' in stock.
8. Cancel the order -> trigger restores stock, payment refunded
9. Kids profile cannot watch adult content
   Rejected -> 'The Boys' is rated A - blocked on a Kids profile.
```

15 Analytical charts

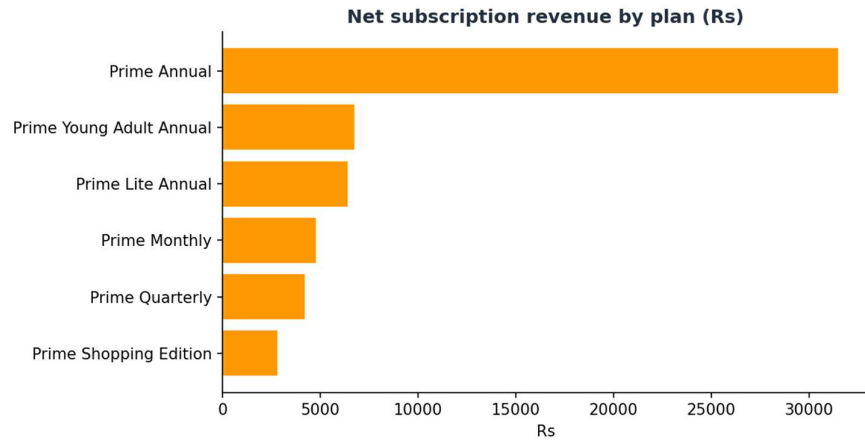


Figure 3 - Net subscription revenue by plan

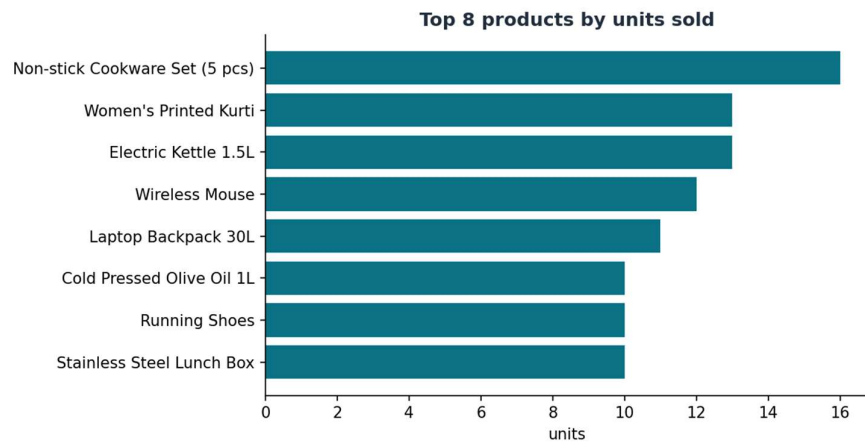


Figure 4 - Best-selling products

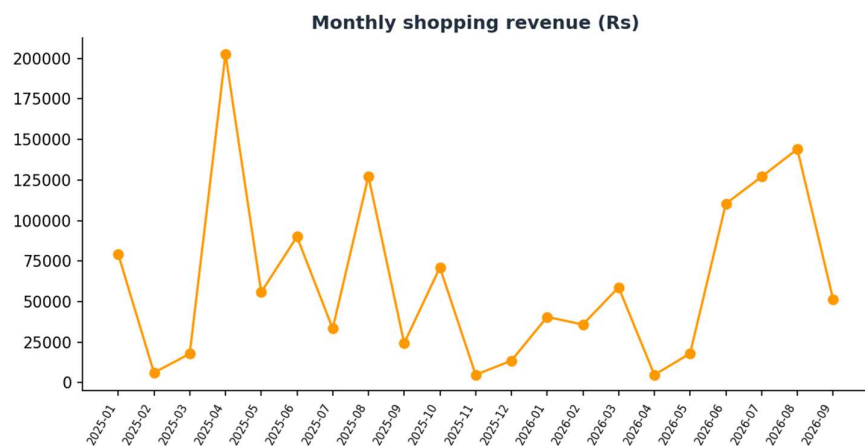


Figure 5 - Monthly shopping revenue

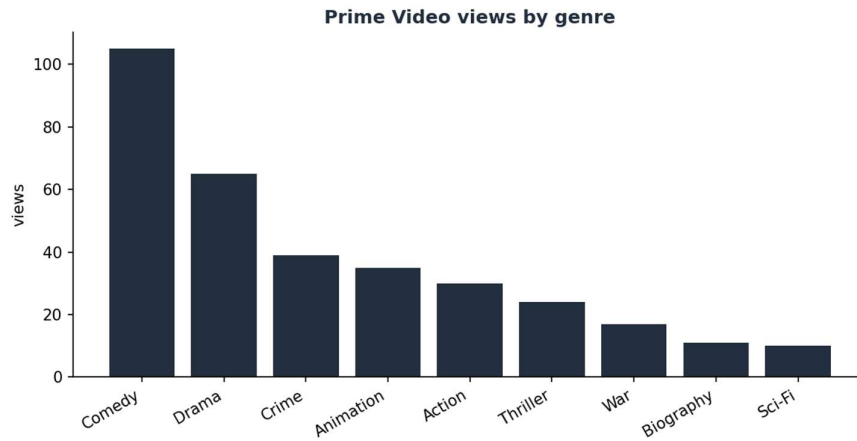


Figure 6 - Prime Video views by genre

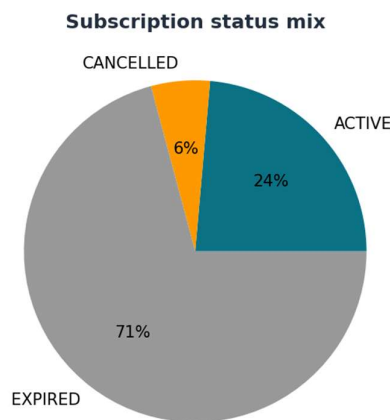


Figure 7 - Subscription status mix

16 Testing

tests/test_database.py contains 21 automated tests (python -m unittest discover -s tests -v). All pass.

Group	What is verified
Schema (9)	24 tables exist; no FK violations in data; UNIQUE e-mail; CHECK on price, rating, dates; FK enforcement; payment exclusive arc; cascade delete
Views & triggers (5)	Order-total view equals manual sum; one-ACTIVE-subscription rule; audit trigger; stock reduce/restore; overselling blocked
Services (7)	Duplicate registration; delivery fee vs Prime; double subscription rejected; failed order rolled back; cancel restores stock and refunds; kids-profile block; review only after delivery

17 Conclusion, limitations and future scope

The project delivers a complete, normalized, constraint-driven database for an Amazon-Prime-like service, together with views, triggers, 25 queries, a transactional Python application and automated tests. It shows how the DBMS itself - not only application code - can guarantee data quality.

Limitations

- Sample data is small and fictional; no real payment gateway; passwords hashed with SHA-256 for demonstration (use bcrypt/argon2 in production).
- No concurrency stress testing; no partitioning or replication.

Future scope

- Web front-end (Flask/Django) over the same services layer.
- Coupons, wishlists, returns and refunds workflow tables.
- Recommendation engine, full-text product search, data warehouse star schema for reporting.
- Role-based access control (GRANT/REVOKE) and encryption of personal data.

18 References

- A. Silberschatz, H. Korth, S. Sudarshan - Database System Concepts, 7th ed., McGraw-Hill.
- R. Elmasri, S. Navathe - Fundamentals of Database Systems, 7th ed., Pearson.
- MySQL 8.0 Reference Manual - <https://dev.mysql.com/doc/>
- SQLite documentation - <https://www.sqlite.org/docs.html>
- Graphviz documentation - <https://graphviz.org/documentation/>
- Python sqlite3 module - <https://docs.python.org/3/library/sqlite3.html>

19 Team contribution

Member	Roll no.	Contribution
[Name 1]		Requirement analysis, ER design
[Name 2]		Schema / DDL, normalization
[Name 3]		Sample data, SQL queries, triggers
[Name 4]		Python application, testing, report